

Exact architecture exploration for heterogeneous edge-cloud cyber-physical systems

Nikita Tarasov^{1*} and Vasyl Tomyuk¹

^{1*}Institute of Computer Science and Information Technologies, Lviv Polytechnic National University, S. Bandery St. 12, Lviv, 79013, Ukraine.

*Corresponding author(s). E-mail(s): dev.nikita@outlook.com;
Contributing authors: vasyl.a.tomiuk@lpnu.ua;

Abstract

Deciding how many edge nodes and gateways a distributed cyber-physical system should have, which hardware and network classes to use, where each processing stage runs and how far it is replicated determines whether the system meets its deadline and what it costs to run. The number of candidate architectures grows into the millions, while a single candidate is evaluated by simulation that can take up to a minute, so the practical budget is the number of expensive evaluations an engineer can afford. Exhaustive search pays for every candidate; evolutionary search fits a budget but cannot say which trade-offs it missed; and the exact methods closest to this problem commonly exploit objectives that accumulate monotonically with each decision, which end-to-end latency does not, because it depends on the utilisation of a tier that several decisions fix jointly. Here we show that exact exploration remains achievable for such objectives. Bounds obtained by relaxing each term of an objective separately, combined with a decision order that makes the queueing term exactly computable on incomplete designs, allow provably safe elimination of infeasible and dominated regions before any evaluation. On spaces of up to 8.3×10^7 architectures the method calls the expensive evaluator **16–63** times and returns the exact set of optimal trade-offs wherever exhaustive enumeration can confirm it. In an independent containerised deployment with the model frozen beforehand, the predicted ordering matches the measured ordering for **88.6–91.3%** of clearly separated architecture pairs.

Keywords: architecture exploration, cyber-physical systems, edge computing, multi-objective optimisation, design automation

1 Introduction

Engineering a distributed cyber-physical system is an architecture decision before it is a software decision. How many edge nodes and gateways to deploy, which hardware class each should be, where each processing stage runs, how far the ingest stage is replicated and which class of network link connects the tiers together determine whether the system meets its deadline, what it costs, how much energy it draws and how much traffic it puts on the network. These decisions interact: adding edge nodes shortens queues but raises cost and idle power, moving analytics to the cloud unloads the edge but adds a wide-area transfer to every request, and replication buys ingest throughput at the price of compute load and synchronisation traffic.

The number of candidate architectures is the product of the decision domains and reaches 10^5 – 10^7 for even a modest set of variables. Exploring such a space systematically is the classical design-space exploration problem, formalised for embedded platforms by the Y-chart methodology[1] and developed over two decades of multi-objective platform exploration[2, 3]. What makes the distributed case different is not the arithmetic but the price of one answer: for distributed cyber-physical systems a single design point is evaluated by simulation, and an industrial methodology built on execution traces of a lithography machine reports up to a minute per design point while explicitly leaving the search strategy out of scope[4]. A survey of the area concludes that scalable exploration technology for such systems is "more or less non-existing"[5], even though the workload and platform models it needs can now be derived automatically from execution traces[6]. The engineering context is active in the opposite direction: orchestration, placement and scaling across the edge–cloud continuum are the subject of a steady stream of recent system designs[7–10], of scheduling heuristics for cyber-physical workloads[11], and of a broader research agenda for the continuum as a whole[12] — all of which assume that a deployment architecture has already been chosen. The engineering consequence is concrete: the budget that matters is the number of expensive evaluations an engineer can afford, not the size of the space.

Two established options each give up something. Exhaustive enumeration returns the exact set of optimal trade-offs but pays the evaluator for every candidate. Population-based search such as NSGA-II[13] fits a fixed evaluation budget but returns an approximation and, crucially for an engineering decision, cannot say which trade-offs it missed. A third and smaller family reasons about *partial* designs and prunes regions that provably contain nothing optimal, either symbolically with constraint solvers[14–17], by composing partial fronts[18], or through bound-based Pareto pruning in the wider optimisation literature[19–21]. Pareto-optimal exploration has also been applied directly to cyber-physical architectures [22, 23] and to deployment choices for distributed inference pipelines[24]. Integer programming achieves the same guarantee for allocation problems in the edge–cloud continuum[25, 26].

The methods in that family that are closest to our problem share one working assumption: that each decision can only add to the objective, so that a partially specified design already carries a valid lower bound on all its completions. A central objective in latency-sensitive distributed cyber-physical systems violates it. End-to-end latency contains a waiting term that depends on the utilisation of a tier, and utilisation is fixed only jointly, by the number of nodes, their class, the replication

factor and the placement policy at once. Adding a node lowers it. There is therefore no monotone accumulation to bound, and the guarantee that makes exact pruning attractive does not transfer.

Here we show that exact exploration is nevertheless achievable for these objectives, and that it makes expensive architecture evaluation tractable rather than merely faster. We construct admissible bounds for contention-dependent objectives by relaxing each additive term of an objective independently over the domains still reachable, and by ordering the decisions so that the determinants of tier utilisation are fixed early — at which point the waiting term stops being relaxed and becomes exactly computable on an incomplete design. Pruning built on these bounds is provably safe, and the exploration returns the exact Pareto front of the analytical model.

The reduction is large enough to change which studies are practical, not merely to speed up existing ones. On architecture spaces of up to 8.3×10^7 candidates the method expands at most 18 082 partial designs and calls the expensive evaluator 16–63 times, while the front it returns is identical, vector for vector, to the one exhaustive enumeration returns wherever exhaustive enumeration is still feasible. An independent containerised edge–gateway–cloud deployment, run with the analytical model frozen beforehand, shows that the ordering the model produces is the ordering the hardware produces for 88.6–91.3% of the architecture pairs it separates by more than 25% within a workload. We report equally clearly where the gains shrink or the method does not apply: the advantage grows with the cost of one evaluation, so with a very cheap evaluator it is bounded by the overhead of the search itself; no admissible bound may be available at all; and near saturation neither the model nor the deployment is stable.

Table 1 Positioning with respect to the closest prior work. “Partial-design pruning” denotes discarding incomplete designs before completing and evaluating them; “contention-aware” denotes objectives that depend on resource utilisation rather than accumulating over decisions.

Work	Year	System class	Obj.	Exact front	Partial-design pruning	Contention-aware	Real measurements
Blickle et al.[27]	1998	embedded syn-thesis	2	no	no	no	no
Lukasiewicz et al.[16]	2008	embedded syn-thesis	n	no	SAT-based	no	no
Neubauer et al.[14]	2018	heterogeneous MPSoC	3	yes	symbolic	no	no
Kouloumpiris et al.[25]	2024	edge/hub/cloud	1	yes (BILP)	solver	no	UAV case
Sedghani et al.[28]	2024	computing con-tinuum	1	no	no	partly	yes
CompDSE[4]	2025	industrial dCPS	perf.	—	none	yes	traces
Kouloumpiris et al.[26]	2026	edge/hub/cloud	2	yes (BILP)	solver	no	case study
This work	2026	heterogeneous edge–cloud CPS	4	yes	bounds	yes	yes

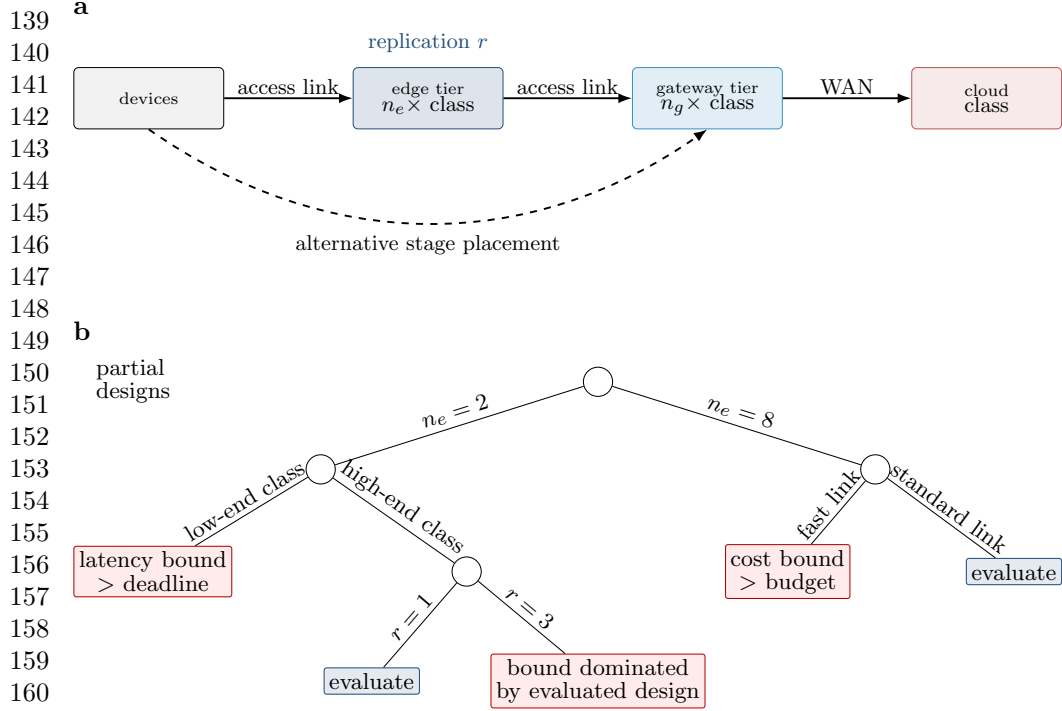


Fig. 1 Exploration by elimination. **a**, Architecture template: the number and class of edge nodes and gateways, the class of the cloud instance, the access link class, the replication factor of the ingest stage and the placement of the processing stages are all design decisions. **b**, Each node of the traversal is a partially specified architecture. It is discarded when an optimistic bound on a constrained quantity already violates its limit (red), or when its optimistic objective vector is dominated by an architecture that has already been evaluated (red); only designs that survive to a leaf reach the expensive evaluator (blue).

2 Results

2.1 Exploration by elimination

HCA-DSE traverses architectures as a tree of partial designs and applies three tests of increasing cost at every node (Fig. 1). A partial design is discarded when it is structurally ill-formed, when an optimistic bound on any constrained quantity already violates its limit, or when an optimistic bound on the whole objective vector is dominated by an architecture that has already been evaluated. Only designs that survive to a leaf are handed to the expensive evaluator. The bounds are admissible by construction: each additive term of an objective is minimised independently over the domains still reachable, and the waiting term becomes exact once the determinants of tier utilisation have been assigned, which the traversal order arranges to happen early (Methods). Safety of the two bound-based rules and preservation of the exact front follow from this property; the statements and proofs are in Methods and Supplementary Note 1.

2.2 Expensive evaluations become the scarce resource that is actually spent

The exploration funnel is reported in Supplementary Table S11 and Fig. 2. On S3 (51 840 candidates), HCA-DSE performs 43–56 full evaluations, removing 99.89–99.92% of the raw space before evaluation. On S4 (345 600 candidates), it performs 52–62 evaluations, a reduction of 99.98–99.98%. Raw-space reductions include structural rejection; Supplementary Table S11 separately reports the smaller number of actual exhaustive evaluator calls. The composition of the funnel differs by workload: for data-intensive sensing the feasibility bounds remove the largest share, whereas for telemetry dominance pruning does most of the work, reflecting which constraint binds in each regime.

The reduction is exact, not approximate. For every scale at which exhaustive enumeration is tractable and for every workload, the set of objective vectors returned by HCA-DSE is identical to the exhaustive Pareto front, with recall and precision of 1.0 (Fig. 3). Admissibility of the bounds was verified exhaustively — all partial designs against all their completions — and for every pruned partial design all completions were enumerated and checked against the claim of the rule that discarded it. The finite-table Z3 baseline recovers the same unrounded analytical front in all six S1/S2 cases (Supplementary Table S5). It consumes 1080 evaluator calls on S1 and 3888 on S2, because the whole structurally valid relation must be built before solving. Total observed runtime is 0.217–4.696 s, including table construction. This establishes an executable exact solver comparison, but does not compare HCA-DSE with a compact symbolic formulation that can avoid evaluator calls.

2.3 The saving converts into time when evaluation is expensive

Whether fewer evaluations translate into less time depends on what an evaluation costs. We measured the overhead of the exploration itself and the cost of one design-point evaluation for two evaluators, and projected the resulting speed-up (Fig. 4). No positive break-even evaluation cost was observed on either scale: with the closed-form evaluator HCA-DSE is already faster than exhaustive enumeration, so the linear crossover model places the threshold below zero and the projections below are gains that grow with evaluator cost rather than gains that start at one. S2: projected speed-up at the measured DES cost is 77.0–147.1 $\times$, with asymptotic evaluation-count ratios of 77.8–149.5 $\times$. S3: projected speed-up at the measured DES cost is 589.4–758.2 $\times$, with asymptotic evaluation-count ratios of 617.1–803.7 $\times$. The DES anchor is 33.6 ms, averaged over 40 designs. The curve in Fig. 4 is a cost projection from measured counts and median overheads; neither exploration strategy was executed end to end with DES in the inner loop. The measured analytical-evaluator speed-up across S2/S3 is 2.58–4.96 $\times$. These hardware- and implementation-specific values do not establish simulator-level Pareto exactness or a universal crossover cost.

2.4 Guarantees at a fraction of a stochastic search budget

Under matched evaluation budgets, NSGA-II approaches the exact front but does not reach it, and cannot certify what is missing (Fig. 5, Table 2). Table 2 reports

231 medians over 20 seeds, with budgets charged to actual full evaluator calls. S2: at
 232 $B = 1000$, NSGA-II median recall is 0.900–0.944 and median HV ratio is 0.999–1.000;
 233 random-search median recall is 0.160–0.188. S3: at $B = 1000$, NSGA-II median recall
 234 is 0.645–0.741 and median HV ratio is 0.969–0.991; random-search median recall is
 235 0.000–0.018. On S3 at $B = 500/1000$, 4 of the six exploratory one-sided comparisons
 236 have unadjusted $p < 0.05$; p ranges from < 0.001 –0.168 and Cliff’s δ from 0.180–0.800.
 237 The individual results are in Supplementary Table S3; no correction for multiple com-
 238 parisons is applied. HCA-DSE recovers the full analytical front on S3 with 43–56
 239 evaluations. All 960 stochastic runs satisfy $N_E = B$ and proposals = N_E + struc-
 240 tural rejections. This accounting includes infeasible full evaluations. Figure 5 shows
 241 seed-level HV distributions; Supplementary Tables S1–S3 retain all budgets. The com-
 242 parison should be read as a statement about guarantees rather than about the quality
 243 of evolutionary search: NSGA-II converges towards the front and would reach it with
 244 more budget, but it cannot certify that anything is missing, whereas HCA-DSE returns
 245 the exact front at a small fraction of the evaluation budget — provided admissible
 246 bounds exist.

247

248 2.5 The mechanism scales because the space is never built

249

250 The scalability sweep (Supplementary Table S12 and Fig. 6) covers raw spaces from
 251 1.3×10^3 to 8.3×10^7 candidates. Across the sweep, expanded partial states never exceed
 252 18082; full evaluations range from 18–63. The largest observed runtime is 5.52s, mea-
 253 sured with Python memory tracing enabled, and peak traced allocations are 75.3 KiB.
 254 This is traced Python memory during search, not process RSS or total interpreter
 255 memory. Exhaustive front equality cannot be checked at these sizes and is not claimed
 256 there.

257 Two internal ablations show where the reduction comes from. The layer ablation
 258 on S3 (Supplementary Table S6) separates the contributions of the three pruning
 259 layers. Structural pruning alone leaves 34 560 full evaluations. Structural plus feasibil-
 260 ity bounds leave 7025–12700, while structural plus dominance bounds leave 328–412.
 261 Combining all layers leaves 43–56. Every variant preserves the exhaustive analyti-
 262 cal front; the separate layers contribute complementary reductions on these spaces.
 263 The layers are complementary, because feasibility pruning removes exactly the sub-
 264 trees in which the dominance test would otherwise be repeated (Supplementary
 265 Table S6). Toggling the contention-aware term of the bound while holding the traver-
 266 sal order fixed isolates its contribution. Supplementary Table S7 crosses traversal
 267 order and queue-bound activation on S3 (all scales in the supplement). The canonical
 268 determinants-early order assigns node counts, node classes, replication and placement
 269 policy before the remaining variables; the determinants-late order assigns replication
 270 and policy last, so the waiting term stays relaxed to zero on every incomplete design.
 271 With the queue bound enabled, the canonical order performs 43–56 full evaluations
 272 against 41–54 for the determinants-late order, while expanding 2375–3221 partial
 273 designs against 4393–5832. Under the canonical order 3777–5139 checked incomplete
 274 designs carry a positive queueing contribution, of which 163–419 are discarded only
 275 because that term is present, against the same archive; under the determinants-late

276

order the count is zero by construction. All 36 scale/workload/order/queue combinations preserve the unrounded exhaustive objective-vector set. Changing the order also changes the archive discovery sequence, so only the within-order on/off comparison isolates the bound toggle. The full order and toggle sweep is in Supplementary Table S7.

2.6 The ordering transfers to a real deployment

To test whether an architecture ranking produced at design time survives contact with a real system, we deployed the same architecture template as containers — one CPU per tier node, per-node link shaping with delay and bandwidth of the design under test, and a wide-area profile on the cloud hop — and measured 36 architectures, three repeats each, 108 runs (Table 3, Fig. 7). The analytical model was frozen before this round: it was not refitted to these observations. Supplementary Table S13 reports analytical mean-latency predictions against the revised DES model. The independent Docker round retains the same 36 configurations and all 108 runs with the analytical model frozen. Table 3 compares predictions with the median of three measured run means. Pooled MAPE is 25.4% and median absolute error is 6.51 ms. Within-workload pair agreement above 25% is 88.6–91.3%, compared with 97.8% over 498 pooled pairs. Pooled agreement benefits from separation between workloads and is not a substitute for within-workload validation. 6 configurations have a ratio of largest to smallest repeated mean latency of at least 1.5; all remain in the main summary. Supplementary Table S4 separately reports the p_{50} comparison and its utilisation/repeat subsets. Differences between predicted means and measured medians must not be interpreted as direct mean-prediction error. Absolute prediction error is therefore substantial, while the decision-relevant quantity — the ordering of architectures that the model separates clearly — transfers. Six configurations were not reproducible across repeats; they are retained in the summary rather than removed, and all sit close to saturation, where neither the analytical model nor the deployment itself is stable.

3 Discussion

For the design spaces studied here, combinatorial growth and costly design-point evaluation together make exhaustive exploration impractical; when the evaluation dominates, reducing the number of evaluator calls is the main available leverage. The results above quantify how much that leverage is worth. Reducing the expensive evaluations from the size of the space to a few dozen, without giving up the guarantee that nothing optimal was discarded, turns a study that would require a simulator campaign into one that fits inside a design meeting; at the per-design-point costs reported for industrial trace-driven exploration[4] the projection is two to three orders of magnitude of wall-clock time.

The reason this works is narrower than "pruning works" and should be stated precisely. Safe pruning of partial designs is an established idea in embedded system synthesis[14, 16] and in multi-objective search more generally[19, 21]. Its usual precondition — objectives that accumulate monotonically with each decision — fails for a central objective in latency-sensitive distributed cyber-physical systems, because

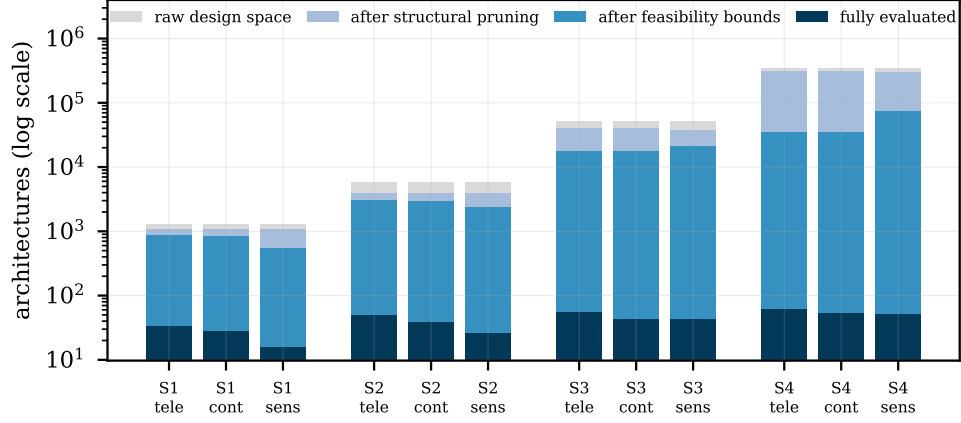


Fig. 2 Architectures are removed before they are evaluated. Raw design space, survivors after structural and after bound-based elimination, and the number of architectures actually handed to the evaluator, for four design-space scales and three workload classes (logarithmic scale).

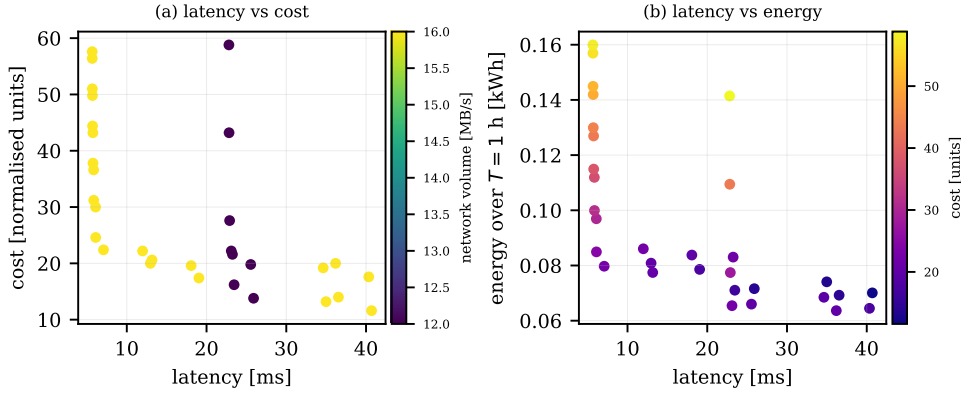


Fig. 3 The recovered set of optimal trade-offs. Exact Pareto front on the 51 840-candidate space for the telemetry workload. **a**, latency versus cost, colour encodes network volume. **b**, latency versus energy over a one-hour horizon, colour encodes cost.

latency depends on tier utilisation and utilisation is fixed jointly by several decisions. The contribution here is a way to keep exactness without that precondition: relax each additive term separately over the domains still reachable, and order the decisions so that the contention term stops being relaxed as early as possible. The mechanism is not specific to the template studied here. Formally, it applies whenever admissible term-wise relaxations can be constructed and a subset of the architectural decisions determines the load-dependent terms. Empirical transfer to a structurally different architecture template has not been demonstrated here, so we make no empirical generality claim beyond the evaluated template. Deployment problems of that shape are

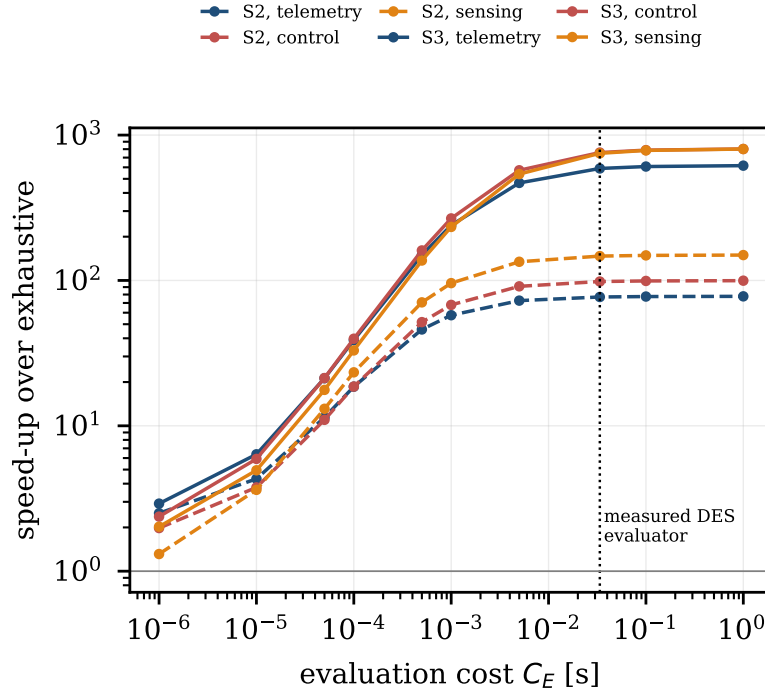


Fig. 4 The saving converts into time when evaluation is expensive. Speed-up over exhaustive enumeration projected from measured evaluation counts and measured overheads as a function of the cost of one design-point evaluation. The dotted line marks the measured cost of the discrete-event evaluator.

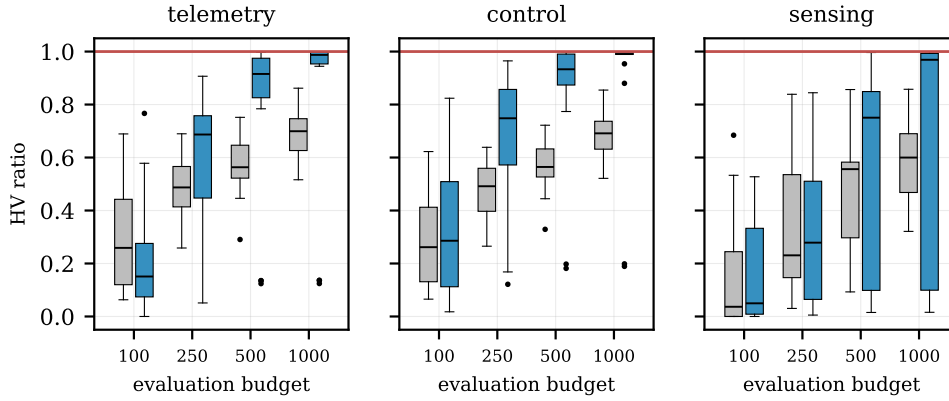


Fig. 5 Budget-matched stochastic exploration. Hypervolume ratio against the exhaustive front on the 51 840-candidate space over 20 seeds. Grey, random search; blue, NSGA-II. The red line marks the exact front recovered by the proposed method.

Table 2 Budget-matched comparison. Medians over 20 seeds at a budget of 1000 full evaluations; the full budget sweep is in Supplementary Tables S1–S3.

scale	workload	method	evals	recall	HV ratio	IGD ⁺	time [s]
S2	telemetry	exhaustive	3 888	1.000	1.000	0.0000	0.060
		random ($B=1000$)	1000	0.185	0.923	0.0451	0.017
		NSGA-II ($B=1000$)	1000	0.944	0.999	0.0003	0.184
		HCA-DSE	50	1.000	1.000	0.0000	0.017
S2	control	exhaustive	3 888	1.000	1.000	0.0000	0.050
		random ($B=1000$)	1000	0.160	0.923	0.0379	0.017
		NSGA-II ($B=1000$)	1000	0.900	0.999	0.0007	0.197
		HCA-DSE	39	1.000	1.000	0.0000	0.015
S2	sensing	exhaustive	3 888	1.000	1.000	0.0000	0.035
		random ($B=1000$)	1000	0.188	0.866	0.0622	0.012
		NSGA-II ($B=1000$)	1000	0.938	1.000	0.0009	0.235
		HCA-DSE	26	1.000	1.000	0.0000	0.012
S3	telemetry	exhaustive	34 560	1.000	1.000	0.0000	0.354
		random ($B=1000$)	1000	0.016	0.699	0.1147	0.013
		NSGA-II ($B=1000$)	1000	0.645	0.987	0.0415	0.199
		HCA-DSE	56	1.000	1.000	0.0000	0.063
S3	control	exhaustive	34 560	1.000	1.000	0.0000	0.323
		random ($B=1000$)	1000	0.018	0.691	0.1248	0.012
		NSGA-II ($B=1000$)	1000	0.648	0.991	0.0244	0.209
		HCA-DSE	43	1.000	1.000	0.0000	0.065
S3	sensing	exhaustive	34 560	1.000	1.000	0.0000	0.356
		random ($B=1000$)	1000	0.000	0.600	0.1747	0.012
		NSGA-II ($B=1000$)	1000	0.741	0.969	0.1670	0.218
		HCA-DSE	43	1.000	1.000	0.0000	0.074

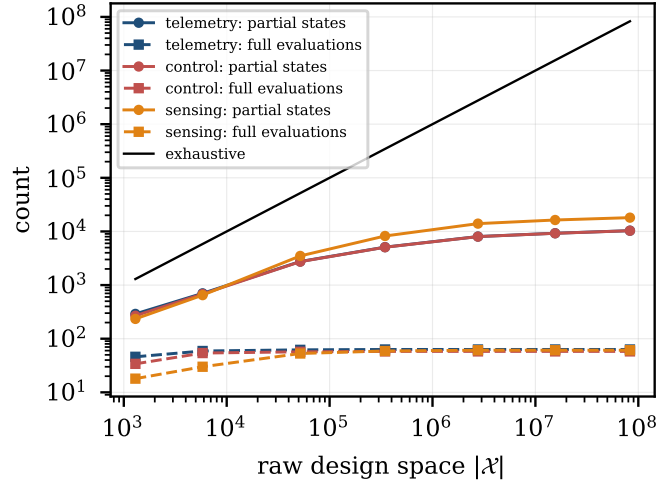


Fig. 6 The design space is never materialised. Expanded partial designs and full evaluations against the raw design-space size, over five orders of magnitude.

Table 3 Independent deployment validation. Analytical predictions against measured run means over 36 architectures and 108 runs with the model frozen beforehand. Pair agreement is the share of architecture pairs whose measured order matches the predicted order among pairs separated by more than 25%. ALL pools workloads.

workload	n	MAPE [%]	Spearman ρ	agreement > 25%	pairs
telemetry	12	17.8	0.776	0.895	38
control	12	12.0	0.865	0.886	44
sensing	12	46.3	0.921	0.913	23
ALL	36	25.4	0.967	0.978	498

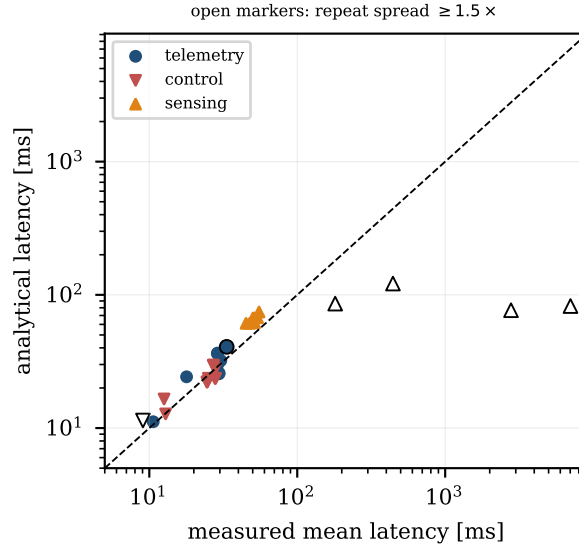


Fig. 7 Predicted against measured latency. Analytical latency versus the median of three measured mean latencies on the independent containerised deployment. Open markers denote architectures whose repeated means differ by a factor of 1.5 or more.

common: placement and scaling across a computing continuum[8, 9], partitioned inference pipelines whose accuracy and robustness depend on where each stage runs[24, 29], and energy-driven co-design of cloud-edge serving[30]. Demonstrating that transfer is the natural next step.

The exactness provided here has a precise scope. The guarantee is that the method returns the exact Pareto front of the analytical model; it says nothing about the model itself. A design-time method can be exact with respect to its own model and still inherit every error of that model. This is why the deployment round matters more than the correlation coefficient it produces: with 25% absolute error, the model is not a substitute for a simulator or a measurement, but it orders clearly separated architectures the same way the hardware does in roughly nine cases out of ten, which

is what a screening stage requires. The productive arrangement is therefore two-level. Analytical bounds remove the bulk of the space exactly and cheaply; the few dozen survivors are where a high-fidelity simulator or a physical deployment should spend its time. Nothing in the method requires the analytical model to be the final word, and an evaluator-centric methodology plugged in behind it would be strictly better than either component alone.

Three limits apply to the results reported here. First, the size of the gain is set by the cost of one evaluation, not by the size of the space. With the closed-form evaluator, which costs microseconds, the measured speed-up is 2.58–4.96×; the projected gains against the discrete-event evaluator are two to three orders of magnitude larger. As the evaluator gets cheaper the search overhead becomes the dominant term and the advantage narrows towards the ratio of overheads, so the method should be chosen on evaluator cost rather than on space size. Second, the guarantee is only as good as the bounds: a bound that cannot be proven admissible must not be used, because an inadmissible bound silently converts an exact method into an approximate one with no signal in the output. Third, near saturation the design-time prediction breaks down and so does the deployment, as six of our measured configurations show; architectures in that regime should be treated as unresolved rather than ranked. Extending the guarantee to objectives that resist closed-form bounds, for example through surrogate bounds that remain provably admissible, is the most valuable open direction we see.

4 Methods

4.1 Architecture template and design space

The template has four tiers — devices, edge nodes, gateways and cloud — and three application stages per request. Pre-processing runs on the edge or gateway tier, aggregation always on the gateway tier, and analytics on the edge tier or in the cloud; which variant is instantiated is a design decision. A compute class is a tuple $(C, M, P^{\text{idle}}, P^{\text{peak}}, c)$ of capacity [MIPS], memory [MB], idle and peak power [W] and normalised cost; a link class is $(B, \tau^{\text{prop}}, e, c)$ of bandwidth [Mb/s], propagation delay, transfer energy [J/MB] and cost per attached node. The gateway-to-cloud connection is fixed infrastructure (200 Mb/s, 20 ms) and is not a design variable. An architecture is

$$x = (n_e, \kappa_e, n_g, \kappa_g, \ell, \kappa_c, r, \pi), \quad (1)$$

with node counts n_e, n_g , classes $\kappa_e, \kappa_g, \kappa_c$, link class ℓ , replication factor r of the ingest stage and placement policy π . The design space is the Cartesian product $\mathcal{X} = D_1 \times \dots \times D_k$, $k = 8$. Four scales (S1–S4) and three workload classes — high-rate telemetry, deadline-dominated real-time control and payload-dominated sensing — are used throughout; their domains and parameters are listed in Supplementary Tables S8–S10.

4.2 Evaluation model

With $t(i)$ the tier executing stage i , N_t its node count and C_t the per-node capacity of its class, the offered load and utilisation are $\Theta_t = \sum_{i:t(i)=t} \Lambda w_i \mu_i$ and $\rho_t = \Theta_t / (N_t C_t)$,

where Λ is the system arrival rate and $\mu_i = r$ for the ingest stage and 1 otherwise. A tier is modelled as one queueing station with the aggregate capacity of its nodes, deterministic service and Poisson arrivals, for which the Pollaczek–Khinchine result gives the waiting term [31, 32], so that

$$L(x) = \sum_{i=1}^3 \frac{w_i}{C_{t(i)}} + \sum_{i=1}^3 \frac{w_i}{C_{t(i)}} \cdot \frac{\rho_{t(i)}}{2(1 - \rho_{t(i)})} + \sum_{(s,\ell) \in \mathcal{H}(x)} \left(\frac{8s}{B_\ell} + \tau_\ell^{\text{prop}} \right), \quad (2)$$

where $\mathcal{H}(x)$ is the set of hops implied by the policy and the replication factor. The middle term is the contention-dependent one. The aggregation is a tractable approximation of a multi-server tier rather than an exact M/D/c representation, kept deliberately because it is the coarsest form that still makes latency depend on utilisation and keeps the bound computable in closed form. Both the discrete-event evaluator and the physical testbed implement per-node servers, so its cost is measured rather than assumed. Energy over a horizon $T = 1$ h combines idle and load-proportional compute power with transfer energy; cost sums node and link coefficients; network volume is the per-second byte flow. Feasibility requires $\rho_t \leq 0.85$ on every used tier, memory and bandwidth to fit, and the latency, cost and energy budgets to hold. Full expressions are in Supplementary Note 2.

4.3 Admissible bounds

Write $\mathcal{C}(p)$ for the completions of a partial design p . A bound LB_i is admissible if $LB_i(p) \leq f_i(x)$ for all $x \in \mathcal{C}(p)$. Two constructions are used.

Term-wise relaxation. If $f_i(x) = \sum_{t \in T_i} \varphi_t(x)$ with $\varphi_t \geq 0$ and $\varphi_t(p) = \min_{x \in \mathcal{C}(p)} \varphi_t(x)$, then $LB_i(p) = \sum_t \varphi_t(p)$ is admissible, because each term is bounded below independently. The minima are taken per term, so the combination realising them need not correspond to any completion; the bound is optimistic but valid. Because the decision domains are finite, the minimum of each term over the reachable domain is precomputed exactly; for the ordered scalar domains this lookup reduces to a domain endpoint, so the bound is evaluated in constant time.

Exactness on determined terms. If the variables a term depends on are already assigned, the term is constant on $\mathcal{C}(p)$ and its exact value may replace the relaxed one, which preserves admissibility and tightens the bound. The decision order places the determinants of tier utilisation — node counts, classes, replication and policy — before the remaining variables, so the waiting term of (2) becomes exact early in the traversal instead of being relaxed to zero. This is what makes a contention-dependent objective usable without assignment monotonicity. If the determinants are fixed and $\rho_t \geq 1$, no completion has finite latency and the bound is ∞ , which is admissible.

4.4 Pruning rules and exactness

Three rules are applied at each node of the prefix tree. *Structural*: the well-formedness predicate σ (gateways do not outnumber edge nodes, replication does not exceed the edge count, replication is undefined for policies that place ingest off the edge tier) depends only on assigned variables and cannot be repaired by later assignments, so

a violation eliminates all completions. It defines the search space rather than following from the constraints, and the exhaustive baseline applies the same predicate. *Feasibility*: if an admissible bound on a constrained quantity exceeds its limit, no completion is feasible. *Dominance*: if an evaluated feasible architecture y satisfies $F(y) \preceq LB(p)$, then every completion of p is dominated by y and none can be Pareto-optimal. Together with admissibility these give front preservation: the non-dominated set returned equals the exact Pareto front of the model in objective space. Proofs are in Supplementary Note 1. Two consequences are used in the experiments: the guarantee is order-independent, so traversal order affects efficiency only; and it is stated in objective space, so architectures sharing an objective vector may be represented once.

4.5 Verification of correctness

Theoretical correctness and implementation correctness are checked separately. Bound admissibility is verified exhaustively on the smallest scale — every partial design against every completion, for all three workloads. For every partial design that the implementation prunes, all of its completions are enumerated and checked against the claim of the rule that discarded it. Equality with the exhaustive front is re-checked on S1–S3 for every pruning variant as a regression test.

4.6 Baselines

Exhaustive enumeration provides ground truth on S1–S3. Random search and NSGA-II[13] (pymoo[33], population 40, SBX crossover, polynomial mutation, integer repair) run at matched budgets of 100, 250, 500 and 1000 full evaluations with 20 seeds each; budgets are charged to actual evaluator calls, including infeasible ones. An independent exact baseline encodes the finite evaluated relation and solves for the non-dominated set with an SMT solver[34]; it requires the full table to be built first and therefore consumes one evaluator call per structurally valid candidate. Recall and precision are computed on sets of objective vectors, hypervolume[35] is normalised by the exhaustive front, and IGD^+ [36] uses that front as reference, following the guidance on indicator choice in ref. 37. Stochastic baselines are compared with the one-sided Mann–Whitney U test[38] and Cliff’s δ [39], reported unadjusted and treated as exploratory.

4.7 Discrete-event evaluator

A higher-fidelity evaluator simulates each design with multi-server tiers, explicit link contention, deterministic service and Poisson arrivals; it plays the role that a discrete-event simulator such as OMNeT++[40] plays in trace-driven industrial methodologies[4]. It serves two purposes: as an internal reference for the accuracy of the analytical model, and as the anchor of the evaluation-cost analysis, at a measured 33.6 ms per design point averaged over 40 designs.

4.8 Containerised deployment

Each edge node, gateway and cloud instance is a separate container limited to one CPU, so that a node behaves as the single server the model assumes at node level. Services are written in Go and share one binary: a request carries a plan of the remaining stages, each node consumes CPU work equal to w_i/C_t and forwards the remainder with the real payload to the next hop. Links are shaped per node with `tc netem` [41] using the delay and rate of the link class under test, with a wide-area profile on the cloud hop; shaping is applied in the forward direction of each hop, matching the one-way transfers counted by the model. Replication is emulated faithfully: r replicas execute the ingest stage in parallel and carry the synchronisation payload, and the remainder of the chain executes once. A load generator produces Poisson arrivals, 900 requests per run of which 150 are discarded as warm-up. Thirty-six architectures — Pareto-optimal, dominated and near-boundary — were selected subject to fitting on one machine (≤ 4 edge nodes, ≤ 2 gateways), and each was run three times. The analytical model was frozen before this round.

4.9 Reproducibility

The implementation is Python 3 with NumPy, SciPy and pymoo; the deployment services are Go under Docker Compose. Every figure and every numeric table in this paper, main and supplementary, is generated from the raw result files by a single script, so no number is transcribed by hand.

Supplementary information. Supplementary Notes 1–2 and Supplementary Tables S1–S13 accompany this paper.

Declarations

Data availability. All result files underlying the figures and tables — exploration funnels, budget sweeps with per-seed metrics, ablations, scalability sweeps, discrete-event comparisons and the 108 measured deployment runs — are archived at Zenodo, <https://doi.org/10.5281/zenodo.22729667>.

Code availability. The exploration method, the analytical and discrete-event evaluators, the baselines, the containerised deployment, the verification tests and the single script that regenerates every display item from the raw results are archived in the same Zenodo record, <https://doi.org/10.5281/zenodo.22729667>, and developed at <https://github.com/Dev-Nikita/eaheccp>.

Author contributions. N.T. (ORCID 0000-0003-1870-3436) designed the method, implemented the software and the deployment, ran the experiments and wrote the manuscript. V.T. (ORCID 0000-0003-4999-7019) contributed to the system model and the experimental design and reviewed the manuscript. Both authors approved the final version and take responsibility for its content.

Use of AI tools. ChatGPT (OpenAI) was used for language editing of author-written text: grammar, wording and readability. It was not used to generate the study

design, the method, the software, the experiments, the results or their interpretation, and it did not generate or alter any data. All edited text was reviewed by the authors, who take full responsibility for the content of this manuscript. No AI tool is listed as an author.

Funding. The authors received no specific funding for this work.

Competing interests. The authors declare no competing interests.

References

- [1] Kienhuis, B., Deprettere, E. F., van der Wolf, P. & Vissers, K. *A methodology to design programmable embedded systems: The Y-chart approach. Embedded Processor Design Challenges*, 18–37 (Springer, 2002).
- [2] Palermo, G., Silvano, C. & Zaccaria, V. Multi-objective design space exploration of embedded systems. *Journal of Embedded Computing* **1**, 305–316 (2005).
- [3] Pimentel, A. D. Exploring exploration: A tutorial introduction to embedded systems design space exploration. *IEEE Design & Test* **34**, 77–90 (2017).
- [4] Saadatmand, F. S., Stefanov, T., González Alonso, I., Pimentel, A. D. & Akesson, B. CompDSE: A methodology for design space exploration of computing sub-systems within complex cyber-physical systems. *IET Cyber-Physical Systems: Theory & Applications* **10**, e70019 (2025).
- [5] Herget, M. *et al.* Design space exploration for distributed cyber-physical systems: State-of-the-art, challenges, and directions. *25th Euromicro Conference on Digital System Design (DSD)*, 632–640 (2022).
- [6] Saadatmand, F. S. *et al.* Automated Derivation of Application Workload Models for Design Space Exploration of Industrial Distributed Cyber-Physical Systems. *2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 1–8 (2024).
- [7] Casini, D., Pazzaglia, P. & Becker, M. Managing real-time constraints through monitoring and analysis-driven edge orchestration. *Journal of Systems Architecture* **163**, 103403 (2025).
- [8] Rodríguez, P., Mateos-Bravo, J., Laso, S., Herrera, J. L. & Berrocal, J. Predictively controlling the computing continuum with distributed energy-aware orchestration. *Journal of Systems Architecture* **175**, 103752 (2026).
- [9] Filippini, F., Lujak, M. & Ciavotta, M. Distributed replica allocation and load balancing for Edge–Cloud FaaS. *Journal of Systems Architecture* **176**, 103787 (2026).

- [10] Ali, M. N., Ullah, I., Imran, M., Salah ud din, M. & Kim, B.-S. Dependency-aware microservices offloading in ICN-based edge computing testbed. *Journal of Systems Architecture* **171**, 103663 (2026). 737
738
739
740
- [11] Ma, L., Zhang, Y., Zhou, J. & Zhang, G. A gene-inspired metaheuristic for scheduling workflow tasks in mobile edge computing-supported cyber-physical systems. *Journal of Systems Architecture* **151**, 103136 (2024). 741
742
743
744
- [12] Sedlak, B. *et al.* Service orchestration in the computing continuum: Structural challenges and vision (2026). URL <https://arxiv.org/abs/2602.15794>. ArXiv:2602.15794. 745
746
747
- [13] Deb, K., Pratap, A., Agarwal, S. & Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* **6**, 182–197 (2002). 748
749
750
751
- [14] Neubauer, K., Wanko, P., Schaub, T. & Haubelt, C. *Exact multi-objective design space exploration using ASPmT*. *Design, Automation & Test in Europe Conference (DATE)*, 257–260 (2018). 752
753
754
755
- [15] Haubelt, C., Neubauer, K., Schaub, T. & Wanko, P. Design space exploration with answer set programming. *KI - Künstliche Intelligenz* **32**, 205–206 (2018). 756
757
758
- [16] Lukasiewicz, M., Glaß, M., Haubelt, C. & Teich, J. *Efficient symbolic multi-objective design space exploration*. *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 691–696 (2008). 759
760
761
762
- [17] Neubauer, K., Beichler, B. & Haubelt, C. Exact design space exploration based on consistent approximations. *Electronics* **9**, 1057 (2020). 763
764
765
- [18] Haubelt, C. & Teich, J. *Accelerating design space exploration using Pareto-front arithmetics*. *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 525–531 (2003). 766
767
768
- [19] Petchrompo, S., Coit, D. W., Brintrup, A., Wannakrairot, A. & Parlikad, A. K. A review of Pareto pruning methods for multi-objective optimization. *Computers & Industrial Engineering* **167**, 108022 (2022). 769
770
771
772
- [20] Stewart, B. S. & White, C. C. Multiobjective A*. *Journal of the ACM* **38**, 775–814 (1991). 773
774
775
- [21] Mandow, L. & Pérez-de-la Cruz, J. L. Multiobjective A* search with consistent heuristics. *Journal of the ACM* **57**, 1–25 (2010). 776
777
778
- [22] Amir, M. & Givargis, T. Pareto optimal design space exploration of cyber-physical systems. *Internet of Things* **12**, 100308 (2020). 779
780
781
782

- 783 [23] Xiao, Y., Oh, C., Lora, M. & Nuzzo, P. *Efficient exploration of cyber-physical sys-*
784 *tem architectures using contracts and subgraph isomorphism. Design, Automation*
785 *& Test in Europe Conference (DATE)*, 1–6 (2024).
786
- 787 [24] Guo, X., Pimentel, A. D. & Stefanov, T. Automated Exploration and Imple-
788 mentation of Distributed CNN Inference at the Edge. *IEEE Internet of Things*
789 *Journal* **10**, 5843–5858 (2023).
790
- 791 [25] Kouloumpiris, A., Stavrinides, G. L., Michael, M. K. & Theodoridis, T. An opti-
792 mization framework for task allocation in the edge/hub/cloud paradigm. *Future*
793 *Generation Computer Systems* **155**, 354–366 (2024).
794
- 795 [26] Kouloumpiris, A., Stavrinides, G. L., Michael, M. K. & Theodoridis, T. A
796 reliability- and latency-driven task allocation framework for workflow applica-
797 tions in the edge–hub–cloud continuum. *Future Generation Computer Systems*
798 **180**, 108414 (2026).
799
- 800 [27] Blickle, T., Teich, J. & Thiele, L. System-level synthesis using evolutionary
801 algorithms. *Design Automation for Embedded Systems* **3**, 23–58 (1998).
802
- 803 [28] Sedghani, H., Filippini, F. & Ardagna, D. SPACE4AI-D: A design-time tool for
804 AI applications resource selection in computing continua. *IEEE Transactions on*
805 *Services Computing* **17**, 4324–4339 (2024).
806
- 807 [29] Guo, X., Jiang, Q., Pimentel, A. D. & Stefanov, T. Model and system robustness
808 in distributed CNN inference at the edge. *Integration* **100**, 102299 (2025).
809
- 810 [30] Wang, J. *et al.* A co-optimization framework toward energy-efficient cloud–edge
811 inference with stochastic computing and precision-compensating NAS. *Journal*
812 *of Systems Architecture* **172**, 103693 (2026).
813
- 814 [31] Kleinrock, L. *Queueing Systems, Volume 1: Theory* (Wiley, New York, 1975).
815
- 816 [32] Harchol-Balter, M. *Performance Modeling and Design of Computer Systems:*
817 *Queueing Theory in Action* (Cambridge University Press, 2013).
818
- 819 [33] Blank, J. & Deb, K. pymoo: Multi-objective optimization in Python. *IEEE*
820 *Access* **8**, 89497–89509 (2020).
821
- 822 [34] de Moura, L. & Bjørner, N. *Z3: An Efficient SMT Solver. Tools and Algorithms*
823 *for the Construction and Analysis of Systems*, Vol. 4963, 337–340 (2008).
824
- 825 [35] Zitzler, E. & Thiele, L. Multiobjective evolutionary algorithms: a comparative
826 case study and the strength Pareto approach. *IEEE Transactions on Evolutionary*
827 *Computation* **3**, 257–271 (1999).
828
- 827 [36] Ishibuchi, H., Masuda, H., Tanigaki, Y. & Nojima, Y. *Modified distance calcu-*
828 *lation in generational distance and inverted generational distance. Evolutionary*

<i>Multi-Criterion Optimization</i> , 110–125 (Springer, 2015).	829
	830
[37] Audet, C., Bignon, J., Cartier, D., Le Digabel, S. & Salomon, L. Performance indicators in multiobjective optimization. <i>European Journal of Operational Research</i> 292 , 397–422 (2021).	831
	832
	833
	834
[38] Mann, H. B. & Whitney, D. R. On a test of whether one of two random variables is stochastically larger than the other. <i>The Annals of Mathematical Statistics</i> 18 , 50–60 (1947).	835
	836
	837
	838
[39] Cliff, N. Dominance statistics: Ordinal analyses to answer ordinal questions. <i>Psychological Bulletin</i> 114 , 494–509 (1993).	839
	840
	841
[40] Varga, A. & Hornig, R. <i>An overview of the OMNeT++ simulation environment. 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)</i> , 1–10 (2008).	842
	843
	844
[41] Hemminger, S. <i>Network emulation with NetEm. Linux Conference Australia (linux.conf.au)</i> (2005).	845
	846
	847
	848
	849
	850
	851
	852
	853
	854
	855
	856
	857
	858
	859
	860
	861
	862
	863
	864
	865
	866
	867
	868
	869
	870
	871
	872
	873
	874